Module 7: Convolutional Neural Networks & Computer Vision

Ultra-Comprehensive Guide to Building Vision AI Systems

Introduction

Computer Vision is the field of AI that enables machines to "see" and understand images. From medical diagnosis to autonomous vehicles to facial recognition, computer vision powers applications that directly impact billions of lives.

Convolutional Neural Networks (CNNs) are the breakthrough that made modern computer vision possible. Unlike fully connected networks that treat images as flat vectors, CNNs exploit the spatial structure of images through **convolutions**, dramatically reducing parameters while improving accuracy.

This module takes you from understanding how images are digitized and filtered, through building complete vision systems, to deploying transfer learning and state-of-the-art architectures.

PART 1: WORKING WITH IMAGES

Image Fundamentals

Digital Image Representation

An image is a 2D array of numbers (pixels):

```
Image = [
  [255, 200, 150, ...],
  [200, 180, 160, ...],
  [150, 160, 140, ...],
  ...
]

Each number = pixel intensity (0-255 for grayscale)
0 = black, 255 = white
```

Color Images (RGB)

Three channels stacked:

```
Red channel:    [R values]    Shape: (height, width)
Green channel:  [G values]
Blue channel:   [B values]

Combined RGB image: Shape (height, width, 3)

Example: (224, 224, 3) = 224×224 pixels with 3 color channels
         = 224 * 224 * 3 = 150,528 numbers total
```

Image as Tensor

```
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

# Load image
img = Image.open('photo.jpg')
img_array = np.array(img)

print(img_array.shape)  # Output: (480, 640, 3)
# 480 pixels height, 640 pixels width, 3 color channels

# Visualize
plt.imshow(img_array)
plt.show()

# Normalize to [0, 1] for neural network
img_normalized = img_array / 255.0
```

Pixel Operations

Brighten image:

```
brightened = img_array + 50  # Add 50 to all pixels
brightened = np.clip(brightened, 0, 255)  # Ensure valid range
```

Flip image:

```
flipped_horizontal = img_array[:, ::-1, :]  # Reverse columns
flipped_vertical = img_array[::-1, :, :]     # Reverse rows
```

Digitization: Sampling and Quantization

Sampling: How many pixels per inch?

```
High-resolution image (small pixels) = More detail but larger file
Low-resolution image (large pixels) = Less detail but smaller file
```

Quantization: How many bits per pixel?

```
8-bit: 2^8 = 256 colors (standard)
24-bit: 2^24 = 16.7 million colors (photographic)
1-bit: 2 colors (black and white)
```

Image Filtering

Convolution Operation (Foundation of CNNs)

Core Idea: Slide a small filter (kernel) over image, compute sum of element-wise products

Example with 3×3 filter:

```
Input image (5×5):        Filter (3×3):
[1  2  3  4  5]          [1  0  -1]
[2  3  4  5  6]          [2  0  -2]
[3  4  5  6  7]          [1  0  -1]
[4  5  6  7  8]
[5  6  7  8  9]

Step 1: Position filter at top-left
[1  2  3]
[2  3  4]
[3  4  5]

Element-wise multiply:
1*1 + 2*0 + 3*(-1) + 2*2 + 3*0 + 4*(-2) + 3*1 + 4*0 + 5*(-1)
= 1 + 0 - 3 + 4 + 0 - 8 + 3 + 0 - 5 = -8

Output[0,0] = -8

Step 2: Slide right by 1 pixel (stride=1)
Step 3: Repeat for entire image
```

Common Filters

Edge Detection (Sobel Filter):

```
[1  0  -1]     Detects vertical edges
[2  0  -2]
[1  0  -1]
```

```
This is a derivative filter
Edges = regions of rapid intensity change
```

Blur Filter (Gaussian):

```
[1  2  1]  /16    Smooths image
[2  4  2]
[1  2  1]

Each output = weighted average of neighborhood
```

Sharpen Filter:

```
[0  -1  0]    Enhances edges
[-1  5  -1]
[0  -1  0]

Subtracts blurred version from original
```

2D Convolution Implementation

```
import numpy as np
from scipy import signal

def convolve_2d(image, kernel):
    """Manual 2D convolution"""
    image_h, image_w = image.shape
    kernel_h, kernel_w = kernel.shape

    # Output dimensions
    out_h = image_h - kernel_h + 1
    out_w = image_w - kernel_w + 1
    output = np.zeros((out_h, out_w))

    for i in range(out_h):
        for j in range(out_w):
            # Extract region
            region = image[i:i+kernel_h, j:j+kernel_w]
            # Element-wise multiply and sum
            output[i, j] = np.sum(region * kernel)

    return output

# Example
image = np.array([[1, 2, 3, 4, 5],
                  [2, 3, 4, 5, 6],
                  [3, 4, 5, 6, 7],
                  [4, 5, 6, 7, 8],
                  [5, 6, 7, 8, 9]], dtype=float)

sobel = np.array([[1, 0, -1],
```

```
                   [2, 0, -2],
                   [1, 0, -1]], dtype=float)

result = convolve_2d(image, sobel)
print(result.shape)  # (3, 3) - reduced from (5, 5)
```

Stride and Padding

Stride: How many pixels to move filter each step

```
Stride=1: Move 1 pixel each step (standard)
Stride=2: Move 2 pixels each step (downsampling)
Stride=3: Move 3 pixels each step (aggressive downsampling)

Output size = (input_size - kernel_size) / stride + 1
```

Padding: Add zeros around image border

```
No padding:     Input (5,5) → Filter (3,3) → Output (3,3)
Padding=1:      Input (7,7) → Filter (3,3) → Output (5,5)
"Same" padding: Output size = Input size

Advantage: Preserve spatial dimensions
Disadvantage: Border pixels used more (may distort)
```

Pooling Operations

Max Pooling: Take maximum value in region

```
Input:            Max Pool 2×2:
[1  2  3  4]      [2  4]
[5  6  7  8]      [10 12]
[9  10 11 12]
[13 14 15 16]

Operation on top-left 2×2:
max(1, 2, 5, 6) = 6

Output size = (input_size - pool_size) / stride + 1
```

Why Pooling?:

1. **Dimensionality reduction**: 2×2 pooling → 4× fewer values
2. **Translation invariance**: Small shifts don't change max
3. **Feature emphasis**: Keep only important values

Other Pooling Options:

```
# Average Pooling
avg(1, 2, 5, 6) = 3.5

# Min Pooling
min(1, 2, 5, 6) = 1

# Stochastic Pooling
Randomly select from region (prevents overfitting)
```

```
from tensorflow import keras

model = keras.Sequential([
    keras.layers.Conv2D(32, kernel_size=(3, 3), input_shape=(28, 28, 1)),
    keras.layers.MaxPooling2D(pool_size=(2, 2)),  # 2×2 max pooling
    keras.layers.Conv2D(64, kernel_size=(3, 3)),
    keras.layers.MaxPooling2D(pool_size=(2, 2)),
])
```

CNN Architectures

LeNet-5: The Pioneer

Architecture (1998):

```
Input (32×32)
  ↓ Conv (6 filters, 5×5) → 28×28×6
  ↓ MaxPool (2×2) → 14×14×6
  ↓ Conv (16 filters, 5×5) → 10×10×16
  ↓ MaxPool (2×2) → 5×5×16
  ↓ Flatten → 400 values
  ↓ Dense (120)
  ↓ Dense (84)
  ↓ Dense (10 softmax)
Output (10 classes)
```

Key Insights:

- Stacking Conv + Pool layers builds hierarchical features
- Early layers detect edges; later layers detect shapes, objects
- Small networks can be effective (only ~60K parameters)

Results: 99.2% accuracy on MNIST handwritten digits

AlexNet: Deep Learning Revolution

Architecture (2012):

```
Input (227×227×3)
  ↓ Conv 96 filters, 11×11 (stride 4) → 55×55×96
  ↓ ReLU, MaxPool 3×3 → 27×27×96
  ↓ Conv 256 filters, 5×5 → 27×27×256
  ↓ ReLU, MaxPool 3×3 → 13×13×256
  ↓ Conv 384 filters, 3×3 → 13×13×384
  ↓ ReLU
  ↓ Conv 384 filters, 3×3 → 13×13×384
  ↓ ReLU
  ↓ Conv 256 filters, 3×3 → 13×13×256
  ↓ ReLU, MaxPool 3×3 → 6×6×256
  ↓ Flatten → 9216 values
  ↓ Dense (4096)
  ↓ ReLU, Dropout (0.5)
  ↓ Dense (4096)
  ↓ ReLU, Dropout (0.5)
  ↓ Dense (1000)
Output (1000 ImageNet classes)
```

Revolutionary Elements:

- **ReLU activation** (instead of sigmoid/tanh)
  - Prevents vanishing gradient
  - Faster training
- **Dropout** (regularization)
  - Prevents overfitting
  - Ensemble effect
- **GPU training**
  - Massively accelerated deep networks
- **Parallel processing**
  - Multiple GPUs for even faster training

Performance: Won ImageNet competition with 85% top-5 accuracy (vs 74% traditional methods)

Impact: Sparked the deep learning revolution!

VGGNet: Simplicity and Depth

Architecture (2014):

```
Key insight: Simple design beats complex design

Use:
- Small 3×3 filters consistently
```

```
- Stack multiple 3×3 convolutions
- Max pooling after groups
- Deep networks (16-19 layers)

Typical flow:
Input (224×224×3)
  ↓ [Conv (64) → Conv (64)] → MaxPool
  ↓ [Conv (128) → Conv (128)] → MaxPool
  ↓ [Conv (256) → Conv (256) → Conv (256)] → MaxPool
  ↓ [Conv (512) → Conv (512) → Conv (512)] → MaxPool
  ↓ [Conv (512) → Conv (512) → Conv (512)] → MaxPool
  ↓ Flatten
  ↓ Dense (4096) → Dense (4096) → Dense (1000)
Output
```

Why multiple small filters?

```
Two 3×3 filters stacked = Same receptive field as one 5×5
But: 2 × (3×3) = 18 parameters
     1 × (5×5) = 25 parameters

Benefit: Fewer parameters + more non-linearity (ReLU between)
```

Advantages:

- Simpler to understand and implement
- Efficient for deeper networks
- Excellent transfer learning base

Disadvantage:

- More parameters than necessary
- Slower inference than modern architectures

ResNet: Skip Connections Revolution

Problem: Very deep networks stop improving (gradient vanishing)

Solution: Skip connections (Residual networks)

```
Traditional:     x → Conv → Conv → y

ResNet:
                  ┌──────────────────┐
                  │                  │
                  x → Conv → Conv ──(+)→ y
                       (identity)

Output = Conv(Conv(x)) + x
```

Why Skip Connections Work:

Instead of learning f(x), network learns residual:

$$f(x) + x$$

During backprop, gradient flows:

```
∂Loss/∂x = ∂Loss/∂output · (∂output/∂residual + ∂output/∂skip)
         = ∂Loss/∂output · (gradients + 1)

The "+1" prevents gradient from vanishing!
```

Results:

- 152-layer ResNet works well (vs 19-layer VGGNet)
- 84.6% ImageNet top-5 accuracy
- Foundation for all modern architectures

```
import tensorflow as tf

# Residual block
def residual_block(x, filters):
    residual = x

    x = keras.layers.Conv2D(filters, (3, 3), padding='same', activation='relu')(x)
    x = keras.layers.Conv2D(filters, (3, 3), padding='same')(x)

    # Skip connection
    x = keras.layers.Add()([x, residual])
    x = keras.layers.Activation('relu')(x)

    return x
```

GoogleNet (Inception): Multi-Scale Features

Key Insight: Extract features at multiple scales simultaneously

```
Inception Module:

Input
  ├─ 1×1 Conv → Output
  ├─ 1×1 Conv → 3×3 Conv → Output
  ├─ 1×1 Conv → 5×5 Conv → Output
  └─ MaxPool → 1×1 Conv → Output

All concatenated depth-wise

Benefit: Capture features at different scales
         5×5 for large patterns
         3×3 for medium patterns
         1×1 for channel processing
```

Transfer Learning in Computer Vision

Transfer Learning Principle

Intuition: Learning to recognize faces helps learning to recognize expressions

Mechanism:

1. Pre-train on huge dataset (ImageNet: 1.2M images, 1000 classes)
2. Learn general visual features (edges, textures, shapes, objects)
3. Transfer to specific task (medical imaging, pet classification)

Fine-tuning Strategies

Strategy 1: Feature Extraction (Frozen Features)

```
Pre-trained model (trained on ImageNet)
  │
  ├─ Conv block 1 → [FREEZE - don't update]
  ├─ Conv block 2 → [FREEZE]
  ├─ Conv block 3 → [FREEZE]
  │
  └─ New Dense layers → [TRAIN]

Training only top layers:
- Fast training (most layers frozen)
- Works well with small datasets
- Risk: Can't adapt to domain differences
```

Strategy 2: Fine-tuning (Partially Frozen)

```
Pre-trained model
  │
  ├─ Conv block 1 → [FREEZE - general features]
  ├─ Conv block 2 → [FREEZE]
  ├─ Conv block 3 → [TRAIN - domain-specific features]
  │
  └─ New Dense layers → [TRAIN]

Training upper layers + new layers:
- Better adaptation to new domain
- More training time
- Needs more data
- Better final performance usually
```

Strategy 3: Full Fine-tuning

```
Pre-trained model
  │
  ├─ Conv block 1 → [TRAIN with very low learning rate]
```

```
   ├─ Conv block 2 → [TRAIN with very low learning rate]
   ├─ Conv block 3 → [TRAIN with low learning rate]
   │
   └─ New Dense layers → [TRAIN with normal learning rate]

Training all layers:
- Maximum adaptation
- Requires large dataset
- High learning rate → destroy pre-trained weights
- Solution: Use very small learning rates (~0.0001)
```

Transfer Learning Implementation

```
from tensorflow import keras
from tensorflow.keras.applications import ResNet50

# Load pre-trained model
base_model = ResNet50(weights='imagenet', include_top=False)

# Freeze early layers (keep pre-trained weights)
for layer in base_model.layers[:50]:
    layer.trainable = False

# Add custom top layers
model = keras.Sequential([
    base_model,
    keras.layers.GlobalAveragePooling2D(),
    keras.layers.Dense(256, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(num_classes, activation='softmax')
])

# Compile with small learning rate
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train
model.fit(X_train, y_train, epochs=10, batch_size=32)
```

Object Detection

Region-Based Detection (R-CNN Family)

Problem: Localize and classify multiple objects in image

Solution: Detect regions of interest, then classify

```
Region Proposal Network (RPN):
Input image
  ↓ Generate ~2000 candidate regions (ROIs)
  ↓ Classify: Object or background?
  ↓ Refine bounding box coordinates
  ↓ Non-maximum suppression (remove overlaps)
Output: Bounding boxes with class labels
```

YOLO: Real-Time Detection

Key Insight: Treat detection as single regression problem

```
Divide image into S×S grid (e.g., 13×13)

For each grid cell, predict:
  - Bounding box (x, y, width, height)
  - Confidence score
  - Class probabilities

Output: S×S×(5+C) tensor
         5 = bbox coords + confidence
         C = number of classes
```

Advantages:

- Real-time performance (can run on video)
- Sees entire image (context awareness)
- Simple and elegant

Example with YOLO v3:

```
import cv2
from yolo_utils import load_model, detect_objects

# Load pre-trained YOLO
model = load_model('yolov3.weights')

# Read image
img = cv2.imread('image.jpg')

# Detect objects
detections = detect_objects(img, model, confidence_threshold=0.5)

# Detections = [[class_id, confidence, x, y, w, h], ...]

# Draw boxes
for detection in detections:
    class_id, conf, x, y, w, h = detection
    cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
    cv2.putText(img, class_names[class_id], (x, y), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 25
```

```
cv2.imshow('Detections', img)
```

Semantic Segmentation

U-Net Architecture

Problem: Pixel-level labeling (which class each pixel belongs to)

Solution: Encoder-Decoder architecture with skip connections

```
Input: (256×256×3)

ENCODER (Downsampling):
  ↓ Conv + Conv → (256×256×64)
  ↓ MaxPool + Conv + Conv → (128×128×128)
  ↓ MaxPool + Conv + Conv → (64×64×256)
  ↓ MaxPool + Conv + Conv → (32×32×512)

DECODER (Upsampling with skip connections):
  ↓ Upsample + Conv + Conv → (64×64×256)  [+ concatenate from encoder]
  ↓ Upsample + Conv + Conv → (128×128×128) [+ concatenate from encoder]
  ↓ Upsample + Conv + Conv → (256×256×64)  [+ concatenate from encoder]
  ↓ Conv (1×1) → (256×256×num_classes)

Output: (256×256×num_classes)
        Each pixel labeled with its class
```

Skip Connections:

```
Why concatenate?
- Decoder loses fine details during pooling
- Skip connections preserve high-res features from encoder
- Concatenated features enable accurate boundary detection
```

U-Net Implementation

```
from tensorflow import keras

def unet(img_size, num_classes):
    inputs = keras.Input(shape=(img_size, img_size, 3))

    # ENCODER
    c1 = keras.layers.Conv2D(64, (3, 3), activation='relu', padding='same')(inputs)
    c1 = keras.layers.Conv2D(64, (3, 3), activation='relu', padding='same')(c1)
    p1 = keras.layers.MaxPooling2D((2, 2))(c1)

    c2 = keras.layers.Conv2D(128, (3, 3), activation='relu', padding='same')(p1)
    c2 = keras.layers.Conv2D(128, (3, 3), activation='relu', padding='same')(c2)
    p2 = keras.layers.MaxPooling2D((2, 2))(c2)
```

```
    # BOTTLENECK
    c3 = keras.layers.Conv2D(256, (3, 3), activation='relu', padding='same')(p2)
    c3 = keras.layers.Conv2D(256, (3, 3), activation='relu', padding='same')(c3)

    # DECODER with skip connections
    u4 = keras.layers.UpSampling2D((2, 2))(c3)
    u4 = keras.layers.Concatenate()([u4, c2])
    c4 = keras.layers.Conv2D(128, (3, 3), activation='relu', padding='same')(u4)
    c4 = keras.layers.Conv2D(128, (3, 3), activation='relu', padding='same')(c4)

    u5 = keras.layers.UpSampling2D((2, 2))(c4)
    u5 = keras.layers.Concatenate()([u5, c1])
    c5 = keras.layers.Conv2D(64, (3, 3), activation='relu', padding='same')(u5)
    c5 = keras.layers.Conv2D(64, (3, 3), activation='relu', padding='same')(c5)

    # OUTPUT
    outputs = keras.layers.Conv2D(num_classes, (1, 1), activation='softmax')(c5)

    model = keras.Model(inputs=[inputs], outputs=[outputs])
    return model

# Use model
model = unet(img_size=256, num_classes=10)
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Transfer Learning Visualization

t-SNE Visualization: Reduce high-dimensional features to 2D for visualization

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

# Extract features from pre-trained model (before classification layer)
feature_extractor = keras.Model(inputs=base_model.input,
                                outputs=base_model.get_layer(-2).output)
features = feature_extractor.predict(X_test)  # Shape: (n_samples, 2048)

# Reduce to 2D using t-SNE
tsne = TSNE(n_components=2, perplexity=30, random_state=42)
features_2d = tsne.fit_transform(features)

# Plot with class colors
for class_id in range(num_classes):
    mask = y_test == class_id
    plt.scatter(features_2d[mask, 0], features_2d[mask, 1],
                label=f'Class {class_id}', alpha=0.6)

plt.legend()
plt.title('Feature Space Visualization (t-SNE)')
plt.show()
```

Advanced Convolutions

Dilated Convolution

Idea: Skip pixels while convolving to increase receptive field

```
Standard convolution (dilation=1):
[x  x  x]  All adjacent pixels
[x  x  x]
[x  x  x]

Dilated convolution (dilation=2):
[x  .  x]  Skip every other pixel
[.  x  .]
[x  .  x]

Receptive field = 5×5 but only 9 parameters (like 3×3)
```

Benefits:

- Large receptive field without increasing parameters
- Useful for dense prediction tasks (segmentation)

Depthwise Separable Convolution

Idea: Separate spatial and channel dimensions

```
Standard Conv:     (3×3×3×32) = 864 parameters
                   Each kernel convolves all 3 channels

Depthwise-Separable Conv:
  ├─ Depthwise:    (3×3×1) × 3 = 27 parameters
  │                Each kernel acts on single channel
  └─ Pointwise:    (1×1×3×32) = 96 parameters
                   Total = 123 parameters (7× fewer!)

Result: Much faster, nearly same accuracy
```

Used in: MobileNet (efficient mobile networks)

1×1 Convolution

Purpose: Mix channel information without changing spatial dimensions

```
Input:  (H × W × C_in)
Filter: (1 × 1 × C_in × C_out)
Output: (H × W × C_out)

Effect: Channel transformation at each pixel
```

```
        Dimensionality reduction
        Parameter-efficient
```

Example: Reduce from 256 channels to 64 channels

Modern Architectures

MobileNet: Efficient Mobile Networks

Goal: Accurate but lightweight networks for mobile devices

Key Techniques:

1. Depthwise-separable convolutions (7× fewer parameters)
2. Width and resolution multipliers (trade accuracy for speed)

```
from tensorflow.keras.applications import MobileNetV2

model = MobileNetV2(input_shape=(224, 224, 3),
                    weights='imagenet',
                    alpha=0.5)  # Alpha: width multiplier (0.5 = half channels)

# Very fast inference (~30ms on mobile)
# Reasonable accuracy (~71% top-1 ImageNet)
```

EfficientNet: Scaling Networks Properly

Key Insight: Balance width, depth, and resolution for optimal efficiency

```
EfficientNet-B0: Baseline (5.3M parameters)
EfficientNet-B1: Larger (7.8M parameters)
...
EfficientNet-B7: Very large (66M parameters)

Scaling rule:
- Depth multiplier: To capture fine features
- Width multiplier: To capture patterns
- Resolution multiplier: To see more
```

Results: State-of-the-art accuracy with fewer parameters than competitors

GPU vs CPU

GPU Advantages

Matrix Operations: CNN = massive matrix multiplications

```
GPU:     1000 cores (simple)  → Parallel processing
CPU:     ~8 cores (complex)   → Sequential processing

Matrix multiply on:
GPU: 2.5 billion operations/sec
CPU: 300 million operations/sec

Speedup: ~8-10× faster on GPU
```

Practical Implications

```
import tensorflow as tf

# Check GPU availability
gpus = tf.config.list_physical_devices('GPU')
print(f"Available GPUs: {len(gpus)}")

# Data on GPU (fast)
# Model on GPU (fast)
# Training much faster

# Without GPU: 1 epoch = 100 seconds
# With GPU:   1 epoch = 5 seconds
```

References

1. Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2023). ImageNet classification with deep convolutional neural networks. Communications of the ACM, 60(6), 84-90.
2. Simonyan, K., & Zisserman, A. (2023). Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556.
3. He, K., Zhang, X., Ren, S., & Sun, J. (2023). Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 770-778).
4. Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., ... & Rabinovich, A. (2023). Going deeper with convolutions. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 1-9).
5. Ronneberger, O., Fischer, P., & Brox, T. (2023). U-Net: Convolutional networks for biomedical image segmentation. In International Conference on Medical image computing and computer-assisted intervention (pp. 234-241). Springer, Berlin, Heidelberg.
6. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2023). You only look once: Unified, real-time object detection. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 779-788).

7. Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., ... & Adam, H. (2023). MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861.
8. Tan, M., & Le, Q. V. (2023). EfficientNet: Rethinking model scaling for convolutional neural networks. In International Conference on Machine Learning (pp. 6105-6114). PMLR.
9. OpenCV Documentation. (2024). Retrieved from https://docs.opencv.org/
10. Stanford CS231n. (2024). Convolutional Neural Networks for Visual Recognition. Retrieved from http://cs231n.stanford.edu/
[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20]

⁂

1. https://www.ibm.com/think/topics/backpropagation
2. https://arxiv.org/html/2408.10006v1
3. https://nlp.stanford.edu/projects/glove/
4. https://towardsdatascience.com/word2vec-glove-and-fasttext-explained-215a5cd4c06f/
5. https://www.ibm.com/think/topics/word-embeddings
6. https://www.geeksforgeeks.org/nlp/word-embeddings-in-nlp/
7. https://www.linkedin.com/pulse/mastering-word-embeddings-nlp-comprehensive-journey-through-sachin-imekc
8. https://www.geeksforgeeks.org/computer-vision/transfer-learning-for-computer-vision/
9. https://journals.plos.org/plosone/article?id=10.1371%2Fjournal.pone.0300767
10. https://nlp2024.jeju.ai/en/week04/index.html
11. https://arxiv.org/abs/2406.10050
12. https://www.geeksforgeeks.org/machine-learning/backpropagation-in-neural-network/
13. https://www.oncologyradiotherapy.com/articles/mcripunet-a-modified-unet-architecture-for-semantic--segmentation-of-images-for-breast-cancer-detection.pdf
14. https://towardsdatascience.com/neural-networks-backpropagation-by-dr-lihi-gur-arie-27be67d8fdce/
15. https://en.wikipedia.org/wiki/Backpropagation
16. https://pub.towardsai.net/backpropagation-2eeb25201095
17. https://viso.ai/deep-learning/convolution-operations/
18. https://www.sciencedirect.com/topics/computer-science/long-short-term-memory-network
19. https://www.linkedin.com/pulse/neural-network-chain-rule-understanding-algorithm-deep-doug-rose-maaie
20. https://ijcat.com/archieve/volume13/issue6/ijcatr13061003.pdf